

CSE 598: Combinatorial Optimization for Graph Algorithms

Zhengjie Min

September 3, 2026

Abstract

This is a template for TCS (and other kind of) lecture notes.

Contents

I	Flow Algorithms	2
1	Introduction	3
1.1	Menger's Theorem	3
1.2	Nash-Williams Theorem	3
1.3	Edmonds' Theorem	4
1.4	Max-Dicut Packing Min-Dijoin Theorem	4
2	Push Relabel for Max Flow	5
2.1	Setting	5
2.2	Definitions	5
A	Algebra	8
B	Analysis	9

Part I

Flow Algorithms

Chapter 1

Introduction

In this part we'll investigate 4 min-max theorems. In these theorems, a max property of a graph is often proved to be equal to a min property of the graph.

We should be aware that such min-max theorems are not seen everywhere. For example:

Claim.

$$\text{Max Matching} \leq \text{Min Vertex Cover}$$

Proof. Let M be a matching and C be a vertex cover. Then each edge in M must have at least one endpoint in C . Since the edges in M are disjoint, we have that $|M| \leq |C|$. ■

The easiest counterexample to show that the above inequality can be strict is a triangle graph. The maximum matching has size 1, while the minimum vertex cover has size 2.

Here we first list the 4 min-max theorems and prove the $\leq$ direction. Then we will prove the tightness in the next few chapters.

1.1 Menger's Theorem

Claim (Menger's Theorem). In a unweighted graph G , the maximum number of edge-disjoint s - t paths $\leq$ the minimum s - t cut (i.e. the minimum number of edges whose removal disconnects s and t).

Proof. Let $P_1, P_2, \dots, P_k$ be edge-disjoint s - t paths. Let C be an s - t cut. Then each path P_i must contain at least one edge in C . Since the paths are edge-disjoint, we have that $k \leq |C|$. ■

1.2 Nash-Williams Theorem

Claim (Nash-Williams Theorem). The maximum number of edge-disjoint spanning trees $\leq \min_P \lfloor \frac{|E(P)|}{|P|-1} \rfloor$ over all partition P of the graph.

Proof. Let $T_1, T_2, \dots, T_k$ be edge-disjoint spanning trees. Let P be a partition of the graph. Then each spanning tree T_i must contain at least $|P| - 1$ edges in $E(P)$ as a spanning tree must touch all the components of the partition P . Since the spanning trees are edge-disjoint, we have that $k(|P| - 1) \leq |E(P)|$, which implies that $k \leq \lfloor \frac{|E(P)|}{|P|-1} \rfloor$. ■

Note. Intuitively we want to find an equality between the maximum number of disjoint spanning trees and the minimum global cut. However one can find the counterexample of it in C_4 , with the maximum number of disjoint spanning trees being 1, while the minimum global cut is 2.

1.3 Edmonds' Theorem

Definition 1.3.1. A s -**arborescence** in a directed graph is a spanning tree rooted at s such that all vertices are reachable from s .

Claim (Rooted Connectivity Theorem). The maximum number of edge-disjoint s -arborescences $\leq$ the minimum s cut (i.e. the minimum number of edges whose removal disconnects s and any vertex).

We won't be focusing on this theorem. Instead, we want to prove a weaker version in the next few chapters:

Definition 1.3.2. A k **bounded in-degree spanning tree** is a spanning tree packing in a directed graph with:

- k edge-disjoint spanning trees $T_1, T_2, \dots, T_k$.
- Let $T = \bigcup_{i=1}^k T_i$. Then for all $t \neq s$, $\text{indeg}_T(t) = k$.

One can verify that the maximum number of edge-disjoint s -arborescences is at most the maximum number of k bounded in-degree spanning trees.

Claim. The maximum number of k bounded in-degree spanning trees $\leq$ the minimum s cut.

Proof. Let the minimum s cut separate a set X from $V \setminus X$, and $s \in V \setminus X$. As every spanning tree can contain at most $|X| - 1$ edges in the subgraph induced by X , we have

$$|E_T(X)| \leq k(|X| - 1)$$

Also, as every vertex in X have in degree k , we have

$$\sum_{v \in X} \text{indeg}_T(v) = k|X|$$

Combining the two inequalities, we have

$$|E(V \setminus X, X)| = k|X| - |E_T(X)| \geq k|X| - k(|X| - 1) = k$$

which implies that the minimum s cut is at least k . ■

1.4 Max-Dicut Packing Min-Dijoin Theorem

Definition 1.4.1. A **dicut** is the edge set $\delta^{\text{out}}(X)$ for any set X that $\delta^{\text{in}}(X) = \emptyset$.

A **dijoin** is a set of edges that, after contraction, makes the graph strongly connected (i.e. every vertex can reach every other vertex).

Claim (Max-Dicut Packing Min-Dijoin Theorem). The maximum number of edge-disjoint dicuts $\leq$ the minimum dijoin.

Proof. Let the maximum number of edge-disjoint dicuts be k . Let $D_1, D_2, \dots, D_k$ be the edge-disjoint dicuts. Let J be a dijoin. Then each dicut D_i must contain at least one edge in J . Since the dicuts are edge-disjoint, we have that $k \leq |J|$. ■

Chapter 2

Push Relabel for Max Flow

2.1 Setting

Definition 2.1.1 ((s, t) -flow problem). $G = (V, E), U : E \rightarrow \mathbb{R}_{\geq 0}, s, t \in V$. An (s, t) -flow is a function $f : E \rightarrow \mathbb{Z}_{\geq 0}$ such that:

- Capacity: $0 \leq f(e) \leq U(e)$ for all $e \in E$.
- Conservation: $f(\delta^{\text{in}}(v)) = f(\delta^{\text{out}}(v))$ for all $v \neq s, t$.

The value of the flow is $\text{val}(f) = f(\delta^{\text{out}}(s)) - f(\delta^{\text{in}}(s))$.

Definition 2.1.2 ((s, t) -cut). An (s, t) -cut is a partition of V into A and B such that:

- $s \in A$ and $t \in B$.
- The capacity (or size) of the cut is $U(\delta^{\text{out}}(A))$.

Theorem 2.1.1. For $B \geq 0$, there's an algorithm that, in $O(mn^2)$ time, either:

- Finds an (s, t) -flow of value B , or
- Finds an (s, t) -cut of size $< B$.

Corollary 2.1.1. The maximum flow is equal to the minimum cut.

Proof. In last chapter, we have shown that the maximum flow is at most the minimum cut. Now we can use binary search to find the maximum flow. If the maximum flow is B , then we can find an (s, t) -flow of value B and an (s, t) -cut of size $< B + 1$. This implies that the minimum cut is at most B , which implies that the maximum flow is equal to the minimum cut.

Alternatively, if max flow is not equal to the min cut, we can find a value between them and run the algorithm. If the algorithm finds a flow of that value, then the max flow is at least that value, which is a contradiction. If the algorithm finds a cut of that value, then the min cut is at most that value, which is also a contradiction. ■

2.2 Definitions

Definition 2.2.1 (Pseudoflow). $f : E \rightarrow \mathbb{Z}_{\geq 0}$ is a **pseudoflow** if it respect the capacity constraints, but not necessarily the conservation constraints.

Definition 2.2.2 (Demand Function). $b : V \rightarrow \mathbb{Z}$ is a **demand function**. f **routes** b if

$$\forall v \in V, f(\delta^{\text{out}}(v)) - f(\delta^{\text{in}}(v)) = b(v).$$

Definition 2.2.3 (Excess). $\text{ex}_f(v) = b(v) + f(\delta^{\text{in}}(v)) - f(\delta^{\text{out}}(v))$ is the **excess** of v . We call a vertex to be **excess** if $\text{ex}_f(v) > 0$ and **deficit** if $\text{ex}_f(v) < 0$.

Appendix

Appendix A

Algebra

Theorem A.0.1. For any polynomial $A \in \mathbb{C}[X]$ of degree $n - 1$, any point set $\{p_0, \dots, p_{n-1}\}$ of size n , $\{(p_i, A(p_i))\}_{i=0}^{n-1}$ determines $A(x)$ and vice versa.

Theorem A.0.2 (Fundamental Theorem of Algebra). Every non-constant single-variable polynomial with complex coefficients has at least one complex root.

In math terms, for $A(x) = \sum_{i=0}^{n-1} a_i x^i$, either $\exists x \in \mathbb{C}$ such that $A(x) = 0$, or $a_0 = a_1 = \dots = a_{n-1} = 0$.

Appendix B

Analysis

Theorem B.0.1 (Master Theorem (simplified)). Let the recurrence relation be:

$$T(n) = aT\left(\frac{n}{b}\right) + n^d$$

where $a \geq 1$ is the number of subproblems, $b > 1$ is the factor by which the problem size is reduced, and $d \geq 0$. The asymptotic bound of $T(n)$ is given by:

1. If $\log_b a > d$, then $T(n) = O(n^{\log_b a})$.
2. If $\log_b a = d$, then $T(n) = O(n^d \log n)$.
3. If $\log_b a < d$, then $T(n) = O(n^d)$.

Theorem B.0.2 (Master Theorem). Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence:

$$T(n) = aT(n/b) + f(n)$$

Then $T(n)$ has the following asymptotic bounds:

- **Case 1:** If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
- **Case 2:** If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some constant $k \geq 0$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.
- **Case 3:** If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n (Regularity Condition), then $T(n) = \Theta(f(n))$.